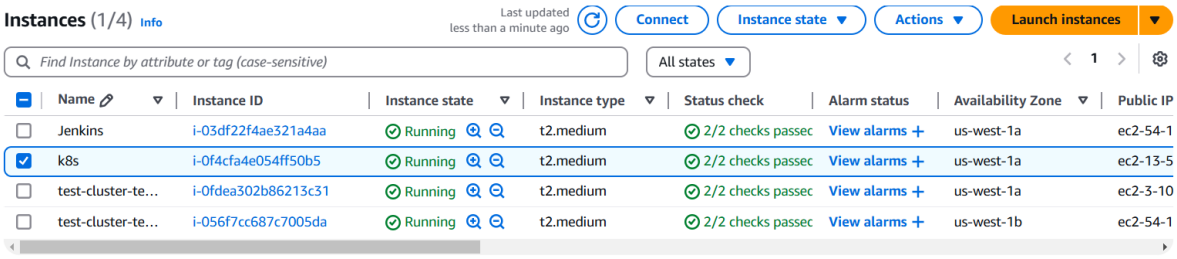
How to connect eks cluster to Jenkins

Eks cluster

- Create a linux server with ubuntu os for setting up eks cluster



Instances (1/4) Info

Last updated less than a minute ago | Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states | 1

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
Jenkins	i-03df22f4ae321a4aa	Running	t2.medium	2/2 checks passed	View alarms +	us-west-1a	ec2-54-1
k8s	i-0f4cfa4e054ff50b5	Running	t2.medium	2/2 checks passed	View alarms +	us-west-1a	ec2-13-5
test-cluster-te...	i-0fdea302b86213c31	Running	t2.medium	2/2 checks passed	View alarms +	us-west-1a	ec2-3-10
test-cluster-te...	i-056f7cc687c7005da	Running	t2.medium	2/2 checks passed	View alarms +	us-west-1b	ec2-54-1

- Connect your ec2 instance with putty or mobaxterm using shh client
- After that you have to install following cli tools

1. **Eksctl**

Description: - eksctl is a simple CLI tool for creating and managing clusters on EKS - Amazon's managed Kubernetes service for EC2. It is written in Go, uses CloudFormation

Command: # for ARM systems, set ARCH to: `arm64`, `armv6` or `armv7`

```
ARCH=amd64
PLATFORM=$(uname -s)_$ARCH

curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$PLATFORM.tar.gz"

# (Optional) Verify checksum
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_checksums.txt" | grep $PLATFORM | sha256sum --check
```

```
tar -xzf eksctl_$PLATFORM.tar.gz -C /tmp && rm eksctl_$PLATFORM.tar.gz

sudo mv /tmp/eksctl /usr/local/bin
```

2. **Kubectl**

Description: - kubectl is the command-line tool for interacting with Kubernetes clusters. It allows users to manage Kubernetes resources and perform various tasks such as deploying applications, inspecting and modifying cluster resources, and troubleshooting cluster issues.

Command: -

- Install kubectl binary with curl on Linux

```
curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
```

- Validate the binary (optional)

Download the kubectl checksum file:

```
curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256
```

- Validate the kubectl binary against the checksum file:

```
echo "$(cat kubectl.sha256)  kubectl" | sha256sum –check
```

- Install kubectl

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

chmod +x kubectl

mkdir -p ~/.local/bin

mv ./kubectl ~/.local/bin/kubectl
```

- Test to ensure the version you installed is up-to-date:

```
kubectl version --client
```

3. **AWS CLI**

Description: - The **AWS Command Line Interface (CLI)** is a tool provided by Amazon Web Services that enables users to interact with AWS services from a terminal or command prompt. It allows you to perform various AWS tasks directly from the command line, including managing resources, configuring settings, and automating workflows.

Command: -

apt install unzip -y

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

unzip awscliv2.zip

sudo ./aws/install

Confirm the installation with the following command: -

aws –version

4. **AWS access key and secret key** you have to provide before create one IAM user with administrator access permission then generate access key and secret key and you configure these keys on your terminal

aws configure (execute this command on your terminal)

```
root@k8s:/home/ubuntu# aws configure
AWS Access Key ID [****************EKUB]: AKIAST6S6Z6L6OSIEKUB
AWS Secret Access Key [****************ZZ/5]: hWsQYQLff4dVYznUqFBK1LgRE7waYazw3L+HZZ/5
Default region name [us-west-1]:
Default output format [None]:
root@k8s:/home/ubuntu#
```

- Now you can create eks cluster by executing following command

eksctl create cluster --name <cluster_name> --region <your_region> --zones <availability_zones> --nodegroup-name <nodegroip_name> --node-type <instance_type> --nodes <no.of.nodes>

- Now your cluster is ready as shown below image 👇

```
2024-11-17 10:04:49 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-cluster"
2024-11-17 10:05:49 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-cluster"
2024-11-17 10:06:49 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-cluster"
2024-11-17 10:06:50 [!]  recommended policies were found for "vpc-cni" addon, but since OIDC is disabled on the cluster, eksctl cannot
configure the requested permissions; the recommended way to provide IAM permissions for "vpc-cni" addon is via pod identity association
s; after addon creation is completed, add all recommended policies to the config file, under `addon.PodIdentityAssociations`, and run `
eksctl update addon`
2024-11-17 10:06:50 [ℹ]  creating addon
2024-11-17 10:06:50 [ℹ]  successfully created addon
2024-11-17 10:06:51 [ℹ]  creating addon
2024-11-17 10:06:51 [ℹ]  successfully created addon
2024-11-17 10:06:52 [ℹ]  creating addon
2024-11-17 10:06:52 [ℹ]  successfully created addon
2024-11-17 10:08:52 [ℹ]  building managed nodegroup stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:08:52 [ℹ]  deploying stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:08:52 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:09:22 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:10:19 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:12:10 [ℹ]  waiting for CloudFormation stack "eksctl-david-k8s-nodegroup-david-node"
2024-11-17 10:12:10 [ℹ]  waiting for the control plane to become ready
2024-11-17 10:12:11 [✔]  saved kubeconfig as "/root/.kube/config"
2024-11-17 10:12:11 [ℹ]  no tasks
2024-11-17 10:12:11 [✔]  all EKS cluster resources for "david-k8s" have been created
2024-11-17 10:12:11 [✔]  created 0 nodegroup(s) in cluster "david-k8s"
2024-11-17 10:12:11 [ℹ]  nodegroup "david-node" has 2 node(s)
2024-11-17 10:12:11 [ℹ]  node "ip-192-168-22-157.ec2.internal" is ready
2024-11-17 10:12:11 [ℹ]  node "ip-192-168-55-13.ec2.internal" is ready
2024-11-17 10:12:11 [ℹ]  waiting for at least 2 node(s) to become ready in "david-node"
2024-11-17 10:12:11 [ℹ]  nodegroup "david-node" has 2 node(s)
2024-11-17 10:12:11 [ℹ]  node "ip-192-168-22-157.ec2.internal" is ready
2024-11-17 10:12:11 [ℹ]  node "ip-192-168-55-13.ec2.internal" is ready
2024-11-17 10:12:11 [✔]  created 1 managed nodegroup(s) in cluster "david-k8s"
2024-11-17 10:12:12 [ℹ]  kubectl command should work with "/root/.kube/config", try 'kubectl get nodes'
2024-11-17 10:12:12 [✔]  EKS cluster "david-k8s" in "us-east-1" region is ready
root@ip-172-31-86-222:/home/ubuntu# kubectl get nodes
NAME                           STATUS   ROLES    AGE     VERSION
ip-192-168-22-157.ec2.internal   Ready    <none>   6m27s   v1.30.4-eks-a737599
ip-192-168-55-13.ec2.internal    Ready    <none>   6m20s   v1.30.4-eks-a737599
root@ip-172-31-86-222:/home/ubuntu#
```

Jenkins

- Connect your ec2 instance with putty or mobaxterm using shh client
- Please make sure you have to install kubectl and aws cli on Jenkins server (you can get commands above where I mentioned for k8s cluster)
- Now install Jenkins on your terminal by executing following commands 👇

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins
```

- Install java on your terminal where are installing Jenkins becz java is prerequisite for Jenkins

sudo apt update

sudo apt install fontconfig openjdk-17-jre

java -version

- Now you have to start your Jenkins by following commands 👇

sudo systemctl enable jenkins

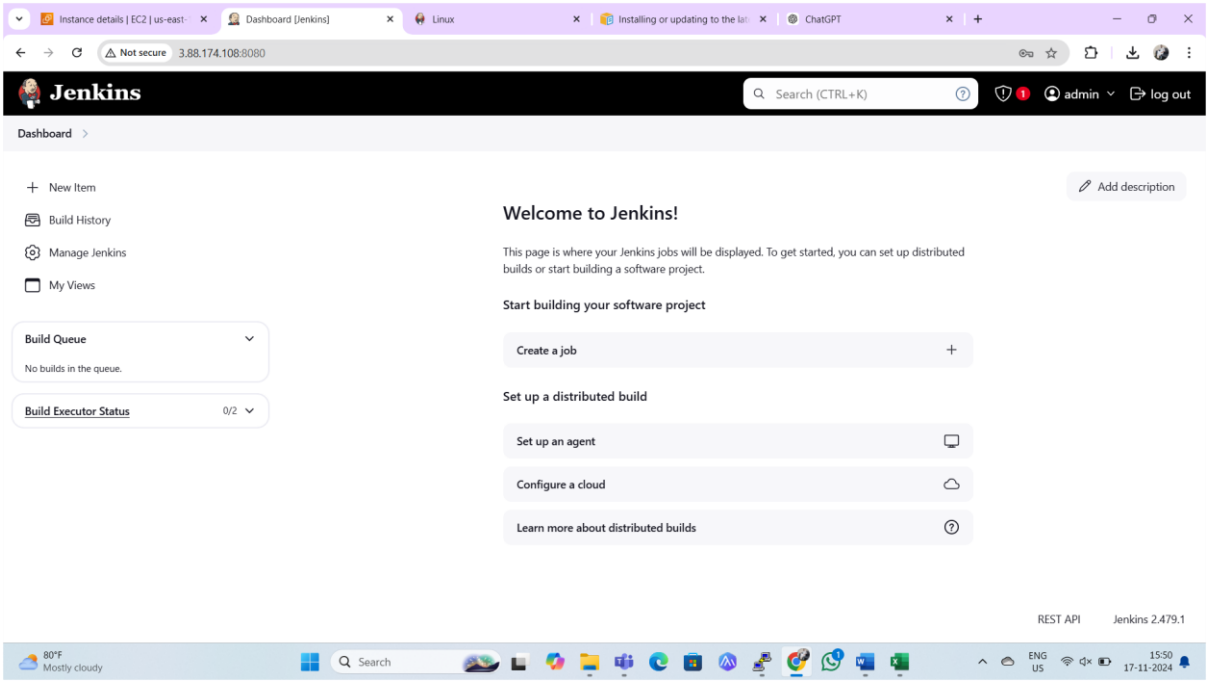
sudo systemctl start Jenkins

sudo systemctl status Jenkins

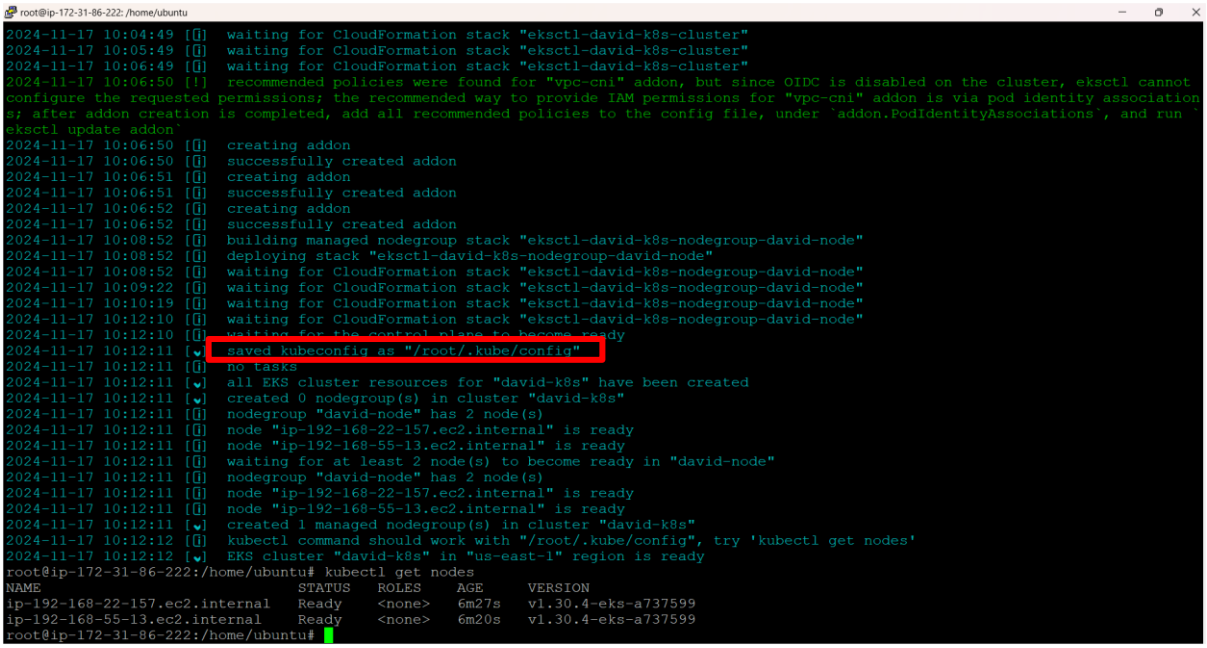
- Now your Jenkins server is ready

```
  inflating: aws/dist/docutils/parsers/rst/include/isoamsn.txt
  inflating: aws/dist/docutils/parsers/rst/include/isomopf.txt
  inflating: aws/dist/docutils/parsers/rst/include/isoamsb.txt
  inflating: aws/dist/docutils/parsers/rst/include/isogrk3.txt
  inflating: aws/dist/docutils/parsers/rst/include/isomopf-wide.txt
  inflating: aws/dist/docutils/parsers/rst/include/s5defs.txt
  inflating: aws/dist/docutils/parsers/rst/include/xhtml1-lat1.txt
  inflating: aws/dist/docutils/parsers/rst/include/isogrk4.txt
  inflating: aws/dist/docutils/parsers/rst/include/isoamso.txt
  inflating: aws/dist/docutils/parsers/rst/include/xhtml1-special.txt
  inflating: aws/dist/docutils/parsers/rst/include/isoamsc.txt
  inflating: aws/dist/docutils/parsers/rst/include/README.txt
  inflating: aws/dist/docutils/parsers/rst/include/isolat2.txt
  inflating: aws/dist/docutils/parsers/rst/include/mmlextra-wide.txt
  inflating: aws/dist/docutils/parsers/rst/include/isoamsr.txt
You can now run: /usr/local/bin/aws --version
root@ip-172-31-86-67:/home/ubuntu# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
     Active: active (running) since Sun 2024-11-17 10:01:55 UTC; 17min ago
   Main PID: 3652 (java)
      Tasks: 45 (limit: 4676)
     Memory: 906.2M (peak: 913.2M)
        CPU: 50.075s
     CGroup: /system.slice/jenkins.service
             └─3652 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPor>

Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.046+0000 [id=214]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.048+0000 [id=212]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=215]        INFO        jenkins.InitReactorRunner$1#on>
Nov 17 10:04:43 ip-172-31-86-67 jenkins[3652]: 2024-11-17 10:04:43.049+0000 [id=71]        INFO        h.m.UpdateCenter$CompleteBatchJ>

root@ip-172-31-86-67:/home/ubuntu#
```

- Access your Jenkins server on browser using instance public ip address with port number as shown below image 👇
- Open port number 8080 in your security group for Jenkins



- Next, we need to copy Kube config file into the Jenkins server from k8s master node as shown below image 👇



- Firstly, we have to provide aws access keys on Jenkins server

```
root@ip-172-31-86-67:/home/ubuntu# aws configure
AWS Access Key ID [None]: AKIARZ5BMVY4PUZ6HXPK
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]:
root@ip-172-31-86-67:/home/ubuntu#
```

- Execute following command on Jenkins server

aws eks update-kubeconfig --name <cluster-name> --region <region>

```
root@ip-172-31-86-67:/home/ubuntu# aws eks update-kubeconfig --name david-k8s --region us-east-1
Added new context arn:aws:eks:us-east-1:124355653176:cluster/david-k8s to /root/.kube/config
root@ip-172-31-86-67:/home/ubuntu#
```

- As seen above image Kube-config file has copied to Jenkins serve from master node i.e. cluster.
- Now execute following commands in Jenkins server one by one

sudo mkdir -p /home/jenkins/.kube

sudo cp /root/.kube/config /home/jenkins/.kube/config

sudo chown -R jenkins:jenkins /home/jenkins/.kube

sudo su - jenkins

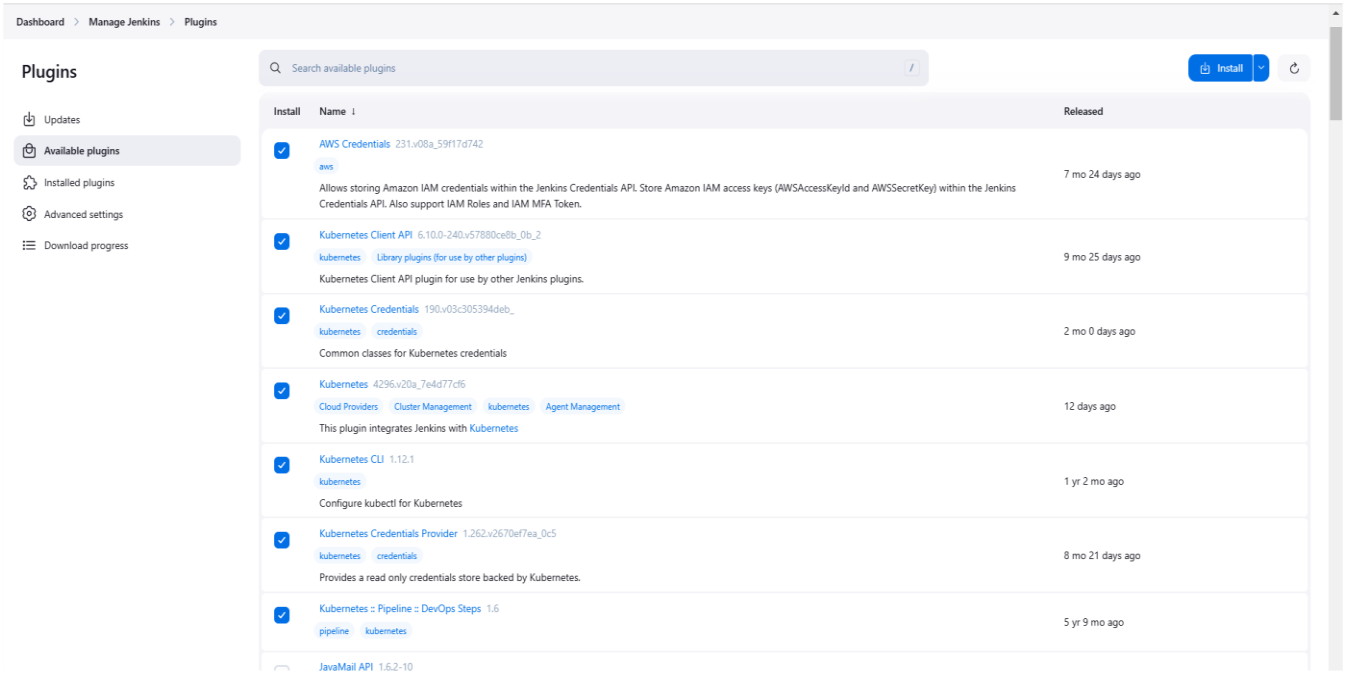
aws configure (provide access keys on Jenkins server)

```
root@ip-172-31-86-67:/home/ubuntu# sudo mkdir -p /home/jenkins/.kube
root@ip-172-31-86-67:/home/ubuntu# sudo cp /root/.kube/config /home/jenkins/.kube/config
root@ip-172-31-86-67:/home/ubuntu# sudo chown -R jenkins:jenkins /home/jenkins/.kube
root@ip-172-31-86-67:/home/ubuntu# sudo su - jenkins
jenkins@ip-172-31-86-67:~$ aws configure
AWS Access Key ID [None]: AKIARZ5BMVY4PUZ6HXPK
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]:
jenkins@ip-172-31-86-67:~$
```

- Install following plugins on Jenkins

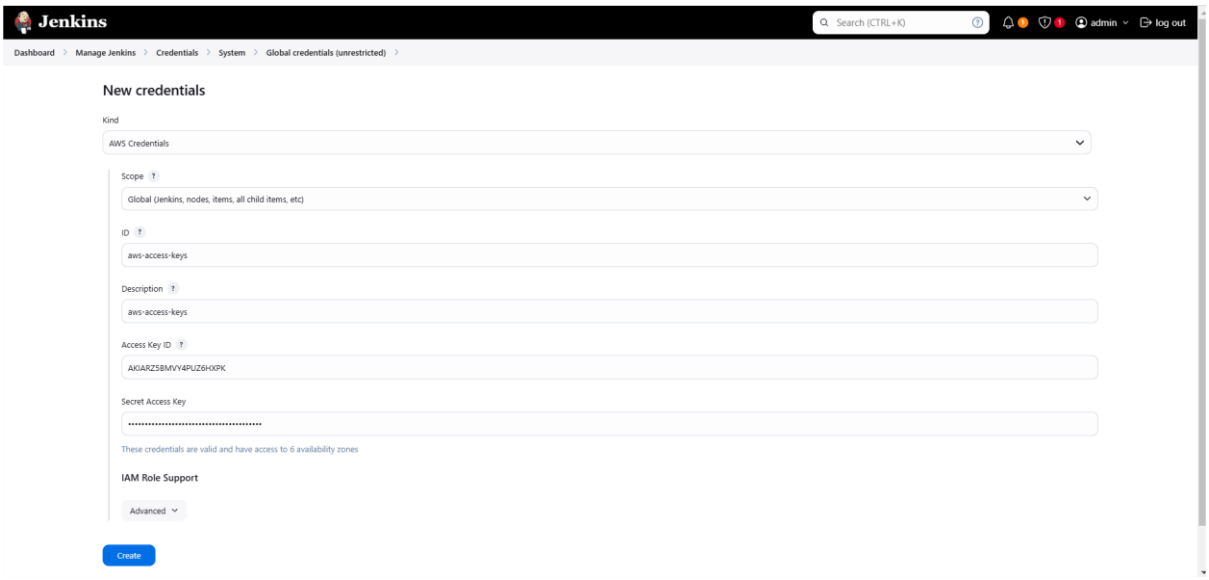
1. AWS Credentials
2. Kubernetes Client API
3. Kubernetes CredentialsVersion
4. Kubernetes

5. Kubernetes CLI
6. Kubernetes Credentials Provider
7. Kubernetes :: Pipeline :: DevOps Steps



- We have to provide aws access keys in Jenkins in credentials

Go to manage Jenkins → go to credentials → system → global credentials



- Now we have to provide Kube-config file in Jenkins web server

- Firstly, we need to copy and save it in a document or file in your local as shown below image
- Execute below two commands on Jenkins server for copying config file

cd /root/.kube

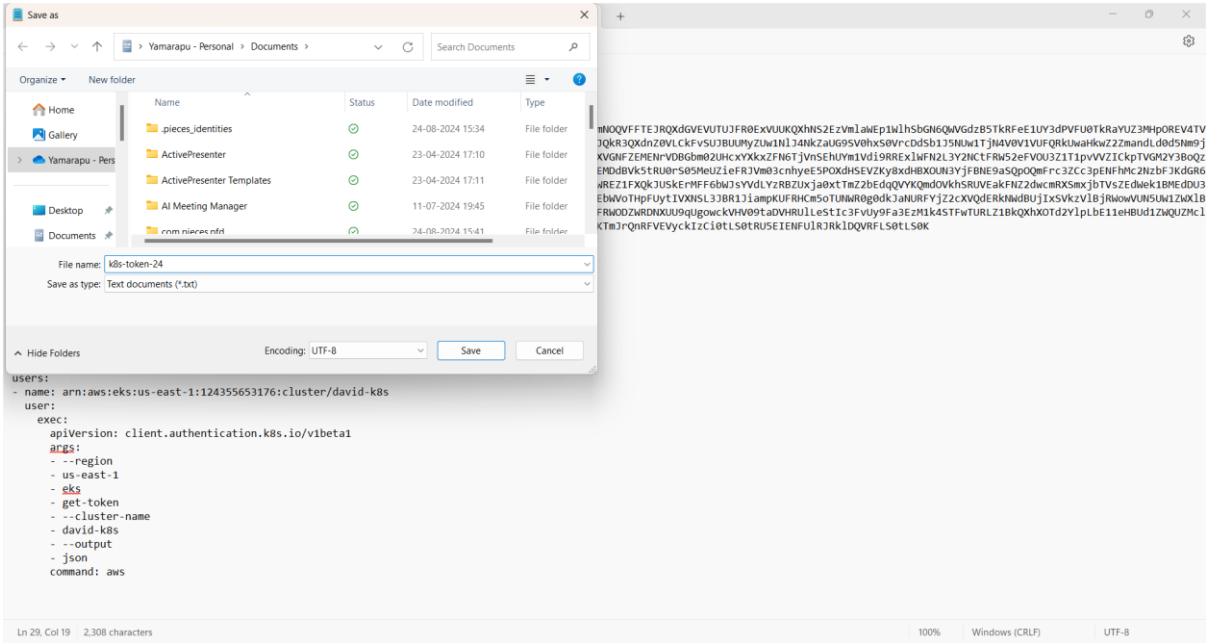
ls

cat config (now copy the entire token and save it in a document or file)

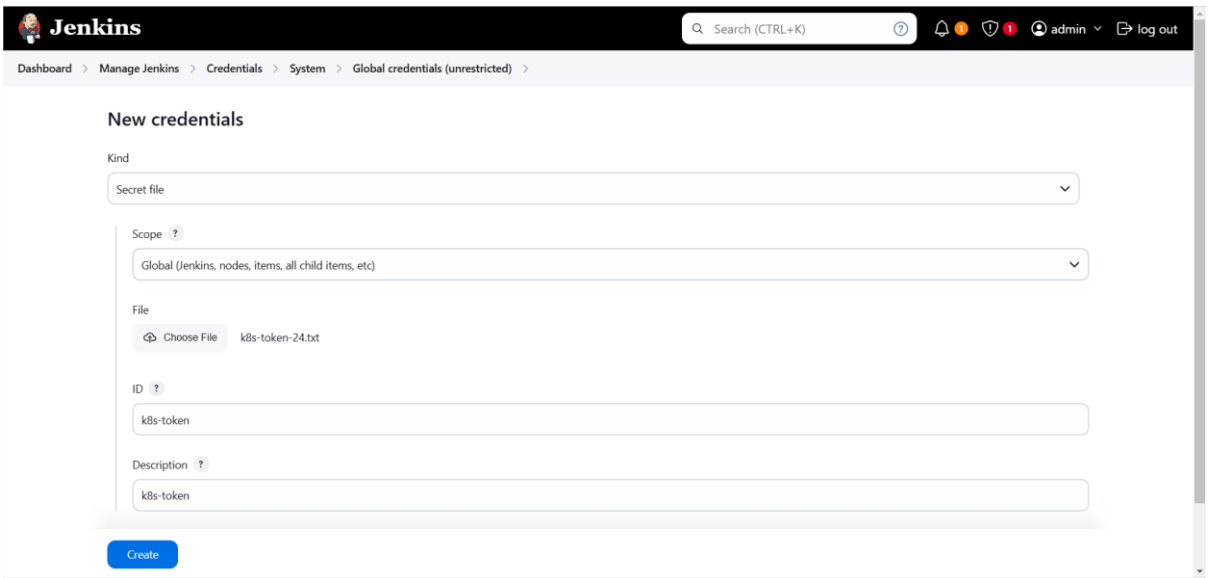
- Please make sure you have to copy this config file from Jenkins server only not from cluster.

```
root@ip-172-31-86-67:/home/ubuntu# cd /root/.kube
root@ip-172-31-86-67:~/.kube# ls
config
root@ip-172-31-86-67:~/.kube# cat config
```

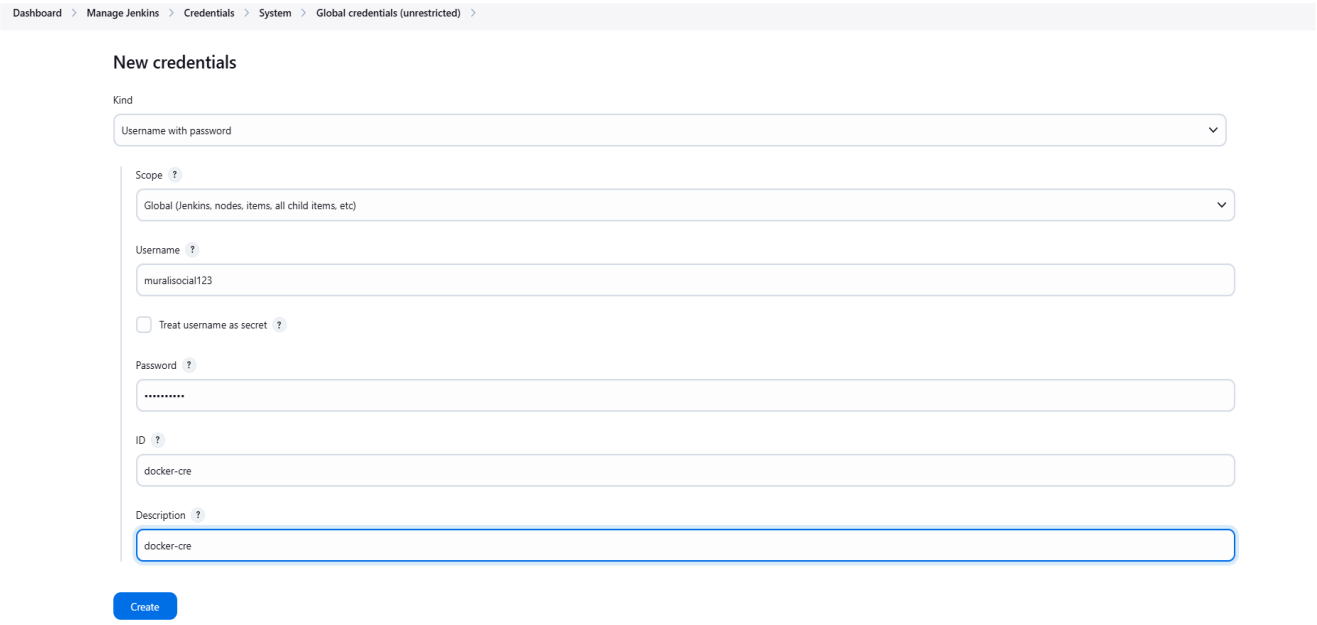

```
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ0lJZHhudlVlMTc0MzR3RFFZSktvWklodmNOQVFFTEJ
RQXdGVEVUTUJFR0ExVUUKQXhNS2EzVmlaWEp1WlhSbGN6QWVGdzB5TkRFeE1UY3dPVFU0TkRaYUZ3MHpOREV4TVRVeE1EQXpORFphTUJVeApFekFSQmdOVkJBTVRDbXQxWW1WeW
JtVjBaWE13Z2dFaU1BMEdDU3FHU01iM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCkFvSUJBUUMyZUw1NlJ4NkZaUG9SV0hxS0VrcDdSb1J5NUw1TjN4V0V1VUFQRkUwaHkwZ2ZmandLd
0d5Nm9jT1UKRmk1aTBKNEtNZFZNL09KVG9yZTFFK2dMQzhyWjY4ZDhlaUFoQ3RSSm1SbjZlU2pJUTdyT3ZqVERieEdKYWkrawo1SXVGNFZEMENrVDBGbm02UHcxYXkxZFN6TjVn
SEhUYm1Vdi9RREx1WFN2L3Y2NCtFRW52eFVOU3Z1T1pvVVZICkpTVGM2Y3BoQzkyakNDZGVCU1ZOOXNGWFdqTG5YRjZCNUsycUw2aVlaNjJrVCtybnJIZDJkbUY1Zzlql3h0cUQ
KZkRHTW45W1RUTUZCTVhEMDdBVk5tRU0rS05MeUZieFRJVm03cnhyeE5POXdHSEVZKy8xdHBXOUN3YjFBNE9aSQpOQmFrc3ZCc3pENFhMc2NzbFJKdGR6d1BKWTNmQWdNQkFBR2
pXVEJYTUE0R0ExVWREd0VCL3dRRUF3SUNwREFQCkJnTlZIUk1CQWY4RUJUQURBUUgvTUIwR0ExVWREZ1FXQkJUSkErMFF6bWJsYVdLYzRBZUxja0xtTmZ2bEdqQVYKQmdOVkhSR
UVEakFNZ2dwcmRXSmxjbTVsZEdWek1BMEdDU3FHU01iM0RRRUJDd1VBQTRJQkFRQUN2TkYxUkdnVQovUVZ0SlpreDBvQ201ckdFZUJJbDQvNno5REhwamhRUW1xZlVSU2lEbWVo
THpFUytIVXNSL3JBR1JiampKUFRHCm5oTUNWR0g0dkJaNURFYjZ2cXVQdERkNWdBUjIxSVkzVlBjRWowVUN5UW1ZWXlBU1R2cVVtVzlESnNPMXZWUGwKNzlIYWhERWJkZm5iWVB
3K09BWHc2ZFRIWDFsSUxtalhacXNwZVhRZnNjcDhYT1QwTmUySFRWODZWRDNXUU9qUgowckVHV09taDVHRUlLeStIc3FvUy9Fa3EzMlk4STFwTURLZ1BkQXhXOTd2YlpLbE11eH
BUd1ZWQUZMclpGSVl3CklUVkRjdk1GOXowQUYyZmNjc0VnSnVBOHlNaGZ2Rlh0R0dIK2Vza201b3N0aE5tNEFJcHpBTElOY0p1RW1WeWgKTmJrQnRFVEVyckIzCi0tLS0tRU5EI
ENFUlRJRklDQVRFLS0tLS0K
    server: https://217AFC0733026373545B6B01FACE5E58.gr7.us-east-1.eks.amazonaws.com
  name: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
contexts:
- context:
    cluster: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
    user: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
  name: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
current-context: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
kind: Config
preferences: {}
users:
- name: arn:aws:eks:us-east-1:124355653176:cluster/david-k8s
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      args:
      - --region
      - us-east-1
      - eks
      - get-token
      - --cluster-name
      - david-k8s
      - --output
      - json
      command: aws
```



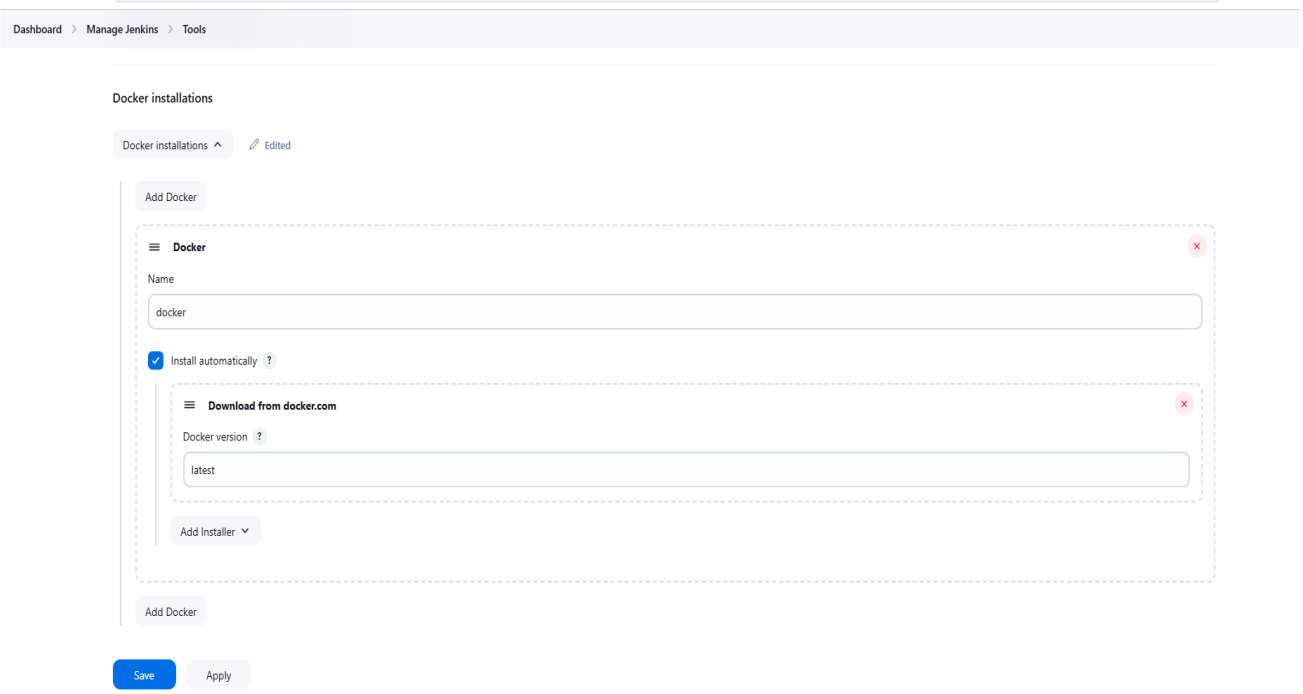
- Now we have to provide this file in Jenkins web server in credentials as follows



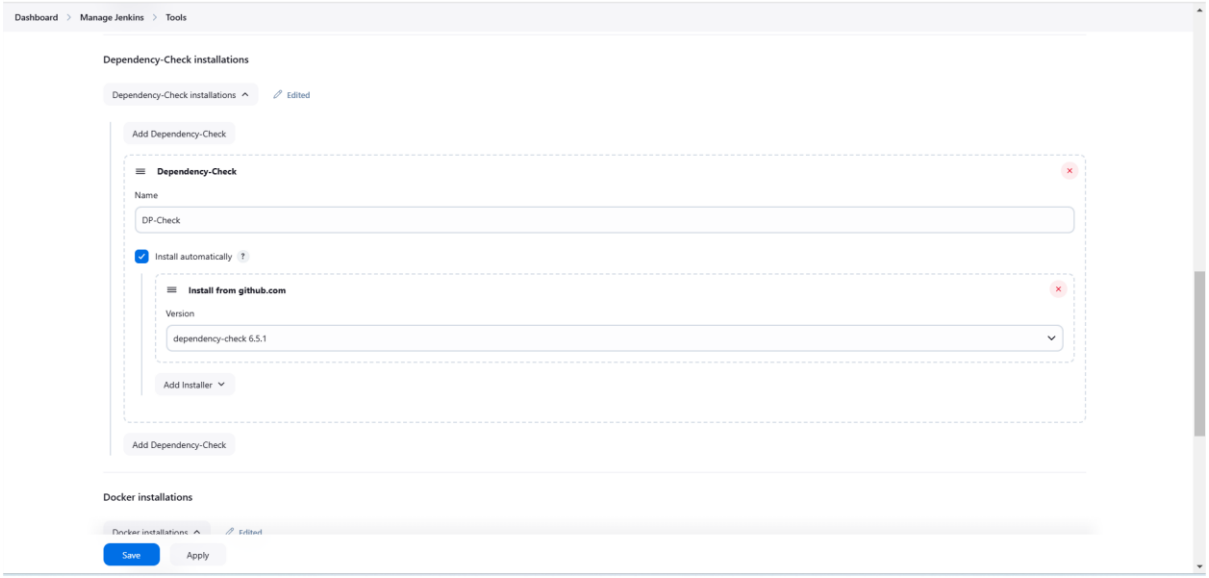
- Provide docker credentials as shown below image 👇



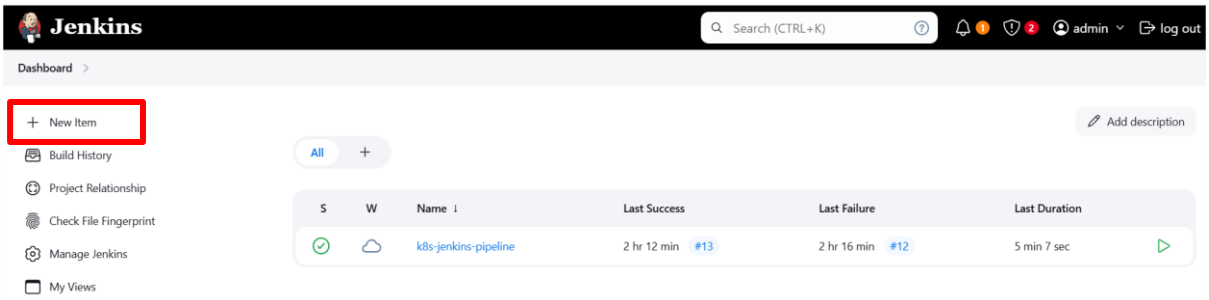
- Configure docker in tools as shown below image 👇



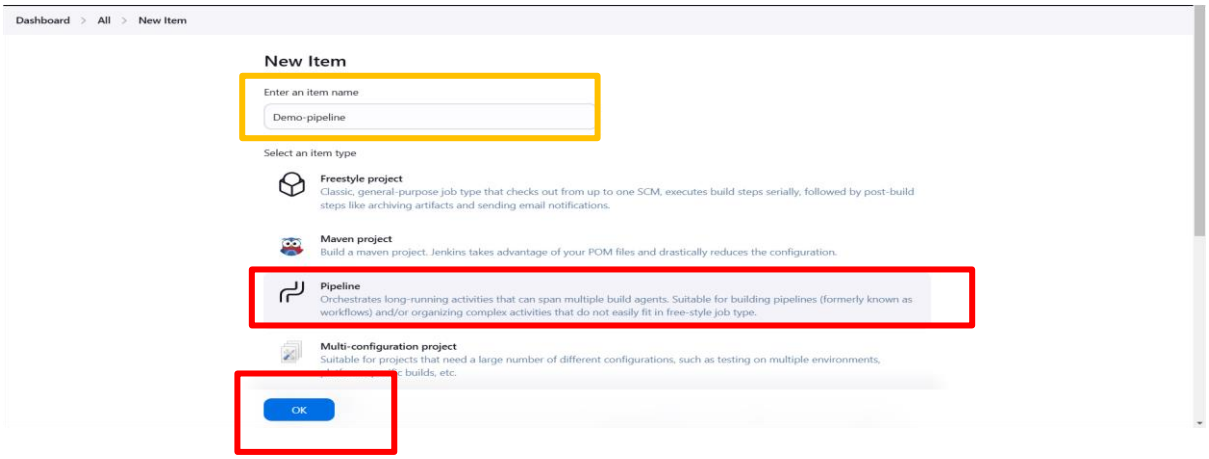
- Configure owasp dependence check as shown below image 👇



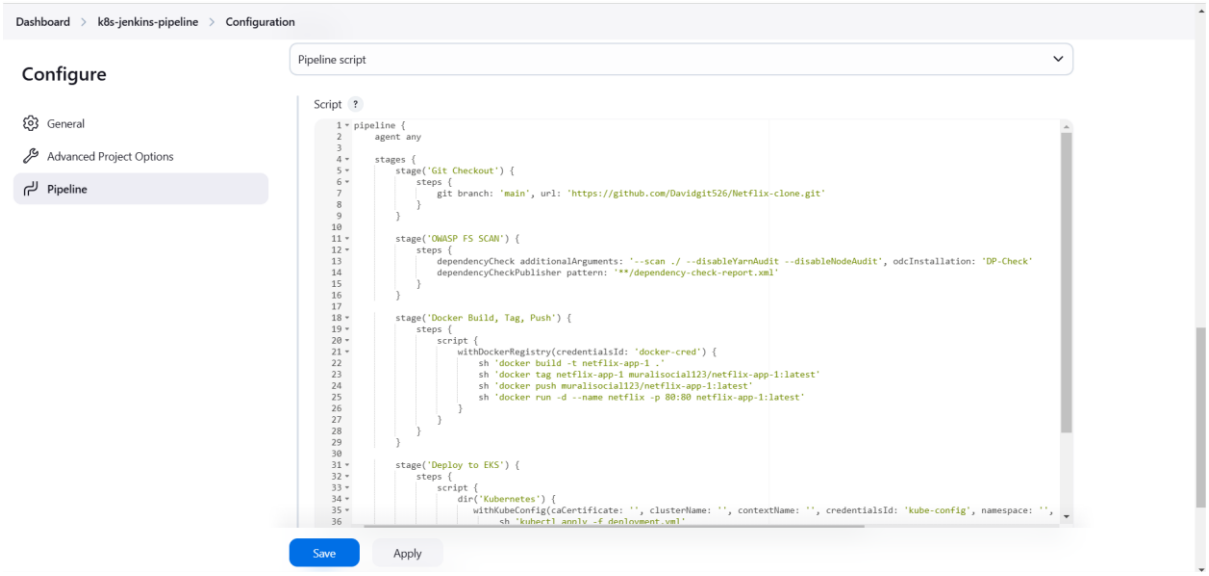
- Create a new item (pipeline) as shown below image 👇



- Click on “New item” for creating pipeline ☝
- Provide a name for pipeline and select pipeline then click on “ok” 👇



- Now create a pipeline by writing script 👇



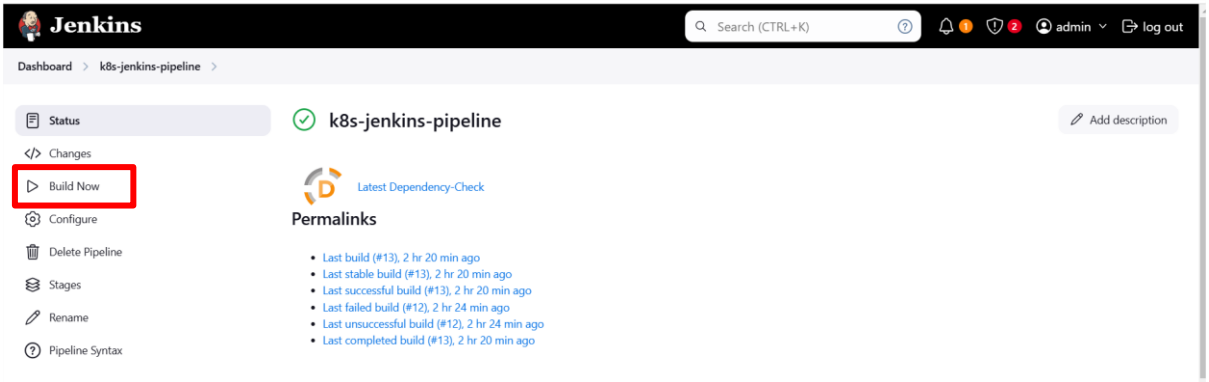
Pipeline Script:

```
pipeline {
    agent any

    stages {
        stage('Git Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/Davidgit526/Netflix-clone.git'
            }
        }

        stage('OWASP FS SCAN') {
            steps {
```

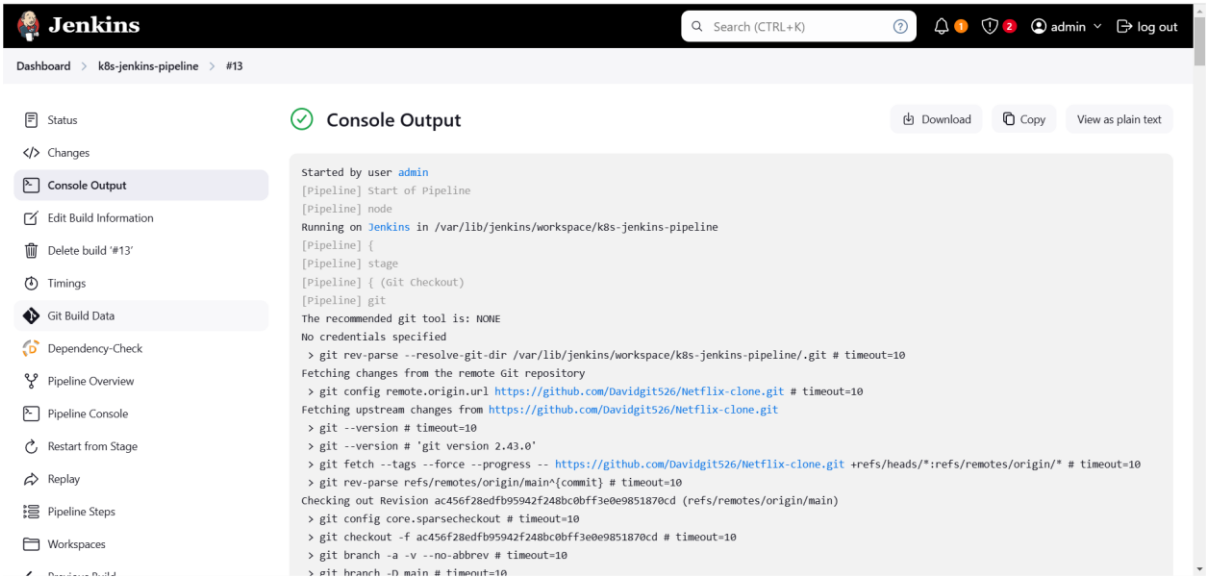
```
            dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit
--disableNodeAudit', odcInstallation: 'DP-Check'
            dependencyCheckPublisher pattern: '**/dependency-check-
report.xml'
        }
    }

    stage('Docker Build, Tag, Push') {
        steps {
            script {
                withDockerRegistry(credentialsId: 'docker-cred') {
                    sh 'docker build -t netflix-app-1 .'
                    sh 'docker tag netflix-app-1 muralisocial123/netflix-app-1:latest'
                    sh 'docker push muralisocial123/netflix-app-1:latest'
                    sh 'docker run -d --name netflix -p 80:80 netflix-app-1:latest'
                }
            }
        }
    }

    stage('Deploy to EKS') {
        steps {
            script {
                dir('Kubernetes') {
                    withKubeConfig(caCertificate: '', clusterName: '', contextName: '',
credentialsId: 'kube-config', namespace: '', restrictKubeConfigAccess: false,
serverUrl: '') {
                        sh 'kubectl apply -f deployment.yml'
                        sh 'kubectl apply -f service.yml'
```

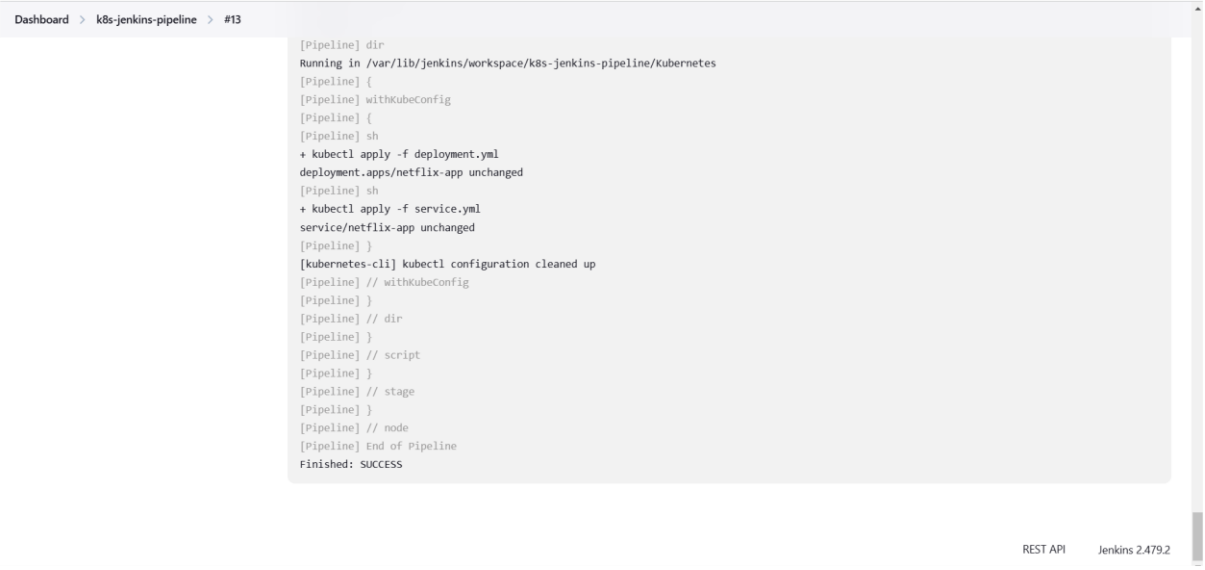
```
                }
            }
        }
    }
  }
 }
}
```

- After saving your script come back to your pipeline and click on “Build Now” to running your pipeline 👇



- Open console output by clicking on build number 👇

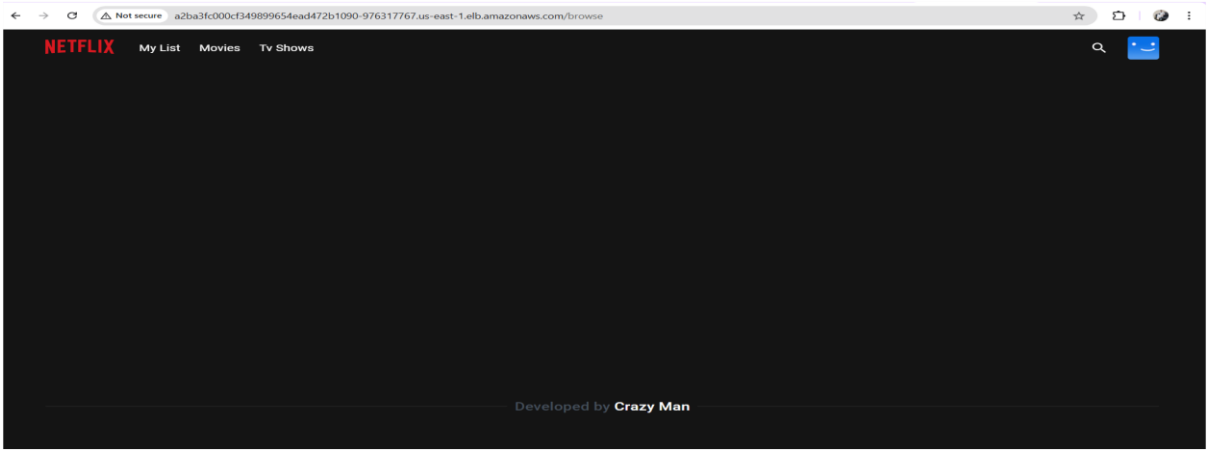




- Go to your cluster check application has deployed or not 👇
- My Netflix application successfully has deployed on eks cluster

```
root@ip-172-31-86-222:/home/ubuntu# kubectl get nodes
NAME                              STATUS   ROLES    AGE     VERSION
ip-192-168-22-157.ec2.internal    Ready    <none>   6m27s   v1.30.4-eks-a737599
ip-192-168-55-13.ec2.internal     Ready    <none>   6m20s   v1.30.4-eks-a737599
root@ip-172-31-86-222:/home/ubuntu# kubectl get all
NAME                                 READY   STATUS    RESTARTS   AGE
pod/netflix-app-648b85589f-58dpz     1/1     Running   0          8m52s
pod/netflix-app-648b85589f-9vrgm     1/1     Running   0          8m52s

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP                                                    PORT(S)
      AGE
service/kubernetes   ClusterIP      10.100.0.1      <none>                                                         443/TCP
      81m
service/netflix-app  LoadBalancer   10.100.196.46   a2ba3fc000cf349899654ead472b1090-976317767.us-east-1.elb.amazonaws.com   80:31638/
TCP   8m50s

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/netflix-app   2/2     2            2           8m52s

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/netflix-app-648b85589f   2         2         2       8m52s
root@ip-172-31-86-222:/home/ubuntu#
```

- Am able to access my application



Monitoring

- Set up Prometheus and Grafana to monitor your application.

Installing Prometheus:

First, create a dedicated Linux user for Prometheus and download Prometheus: